

Комплект лабораторних робіт з архітектури ARM Cortex-M4

Лабораторні роботи №1-3: QEMU, GAS, GDB, завантажувач і Semihosting

Студент

Джулай Іван Олексійович

Група

ІО-331

Номер залікової книжки

1

Загальний опис комплекту

Комплект підготовлено для виконання лабораторних робіт з архітектури ARM Cortex-M4. Практична частина орієнтована на мікроконтролер STM32F407VG, GNU Assembler, тулчейн arm-none-eabi, емулятор QEMU та відлагоджувач GDB.

Для цього комплекту використано номер залікової книжки 1. Отже, для лабораторної роботи №2 обрано варіант 1, а для лабораторної роботи №3 також обрано варіант 1.

Лабораторна робота №1

Тема

Створення мінімального програмного проекту мовою асемблера для ARM Cortex-M4.

Мета роботи

Навчитися створювати мінімальний проект мовою асемблера, виконувати його збірку за допомогою arm-none-eabi-gcc, формувати ELF- і BIN-файли та запускати програму в середовищі QEMU з підключенням GDB.

Хід виконання

Створено файл start.S, який містить таблицю векторів та обробник апаратного скидання.

Створено файл lscript.ld, де задано області FLASH і RAM, а також початкову адресу стеку.

Створено Makefile для автоматизованої збірки, перетворення firmware.elf у firmware.bin та запуску QEMU.

Після запуску QEMU програма очікує підключення GDB через порт tcp::1234.

Фрагмент start.S

```
.syntax unified
.cpu cortex-m4
.thumb

.global vtable
.global __hard_reset__

.type vtable, %object
.type __hard_reset__, %function

vtable:
    .word __stack_start
    .word __hard_reset__ + 1
    .size vtable, . - vtable
```

```

__hard_reset__:
    ldr r0, =__stack_start
    mov sp, r0
loop:
    b loop
    .size __hard_reset__, . - __hard_reset__

```

Результат виконання

У результаті збірки мають бути сформовані файли start.o, firmware.elf і firmware.bin. Під час налагодження GDB завантажує символи з firmware.elf, після чого доступне покрокове виконання програми.

Висновок

У лабораторній роботі створено базовий асемблерний проект для Cortex-M4, описано початковий вектор скидання, задано карту пам'яті та підготовлено автоматизовану збірку проекту.

Лабораторна робота №2

Тема

Основні інструкції 32-бітного ARM процесора для мікроконтролерів.

Мета роботи

Навчитися використовувати арифметичні, логічні та умовні інструкції ARM Cortex-M4, організовувати підпрограми та перевіряти результат виконання через регістри у GDB.

Визначення варіанта

Варіант визначається за формулою: номер залікової книжки % 5 = номер варіанта.

Підстановка значення: 1 % 5 = 1. Для виконання обрано варіант 1: $(a - b) * 3 + 2$.

Варіант	Вираз	Регістр результату
0	$(a + b) / 2 + c!$	r3
1	$(a - b) * 3 + 2$	r3
2	$((a \& b) >> 1) + c!$	r3
3	$(a + b) / c$ або $(a - b) / c$ залежно від знака $a - b$	r3
4	$(a b) * c$ або $(a * b) * c$ залежно від умови $a * b$	r3

Хід виконання

У файлі lab2.S задано константи $a = 14$, $b = 6$, $c = 4$. Для варіанта 1 виконується вираз $(a - b) * 3 + 2$. Після підстановки тестових значень отримуємо: $(14 - 6) * 3 + 2 = 26$.

Параметр збірки для роботи: LAB2_VARIANT=1.

Фрагмент lab2.S

```

.syntax unified
.cpu cortex-m4
.thumb

```

```
.global lab2
```

```
#ifndef LAB2_VARIANT
#define LAB2_VARIANT 1
#endif
```

```
#define A_VALUE #14
#define B_VALUE #6
#define C_VALUE #4
#define SHIFT_VALUE #1
```

```
lab2:
```

```
    push {lr}
    mov r0, A_VALUE
    mov r1, B_VALUE
    mov r2, C_VALUE
```

```
#if LAB2_VARIANT == 0
    //  $r3 = (a + b) / 2 + c!$ 
    add r3, r0, r1
    asr r3, r3, #1
    mov r0, r2
    bl factorial
    add r3, r3, r0
```

```
#elif LAB2_VARIANT == 1
    //  $r3 = (a - b) * 3 + 2$ 
    sub r3, r0, r1
    mov r4, #3
    mul r3, r3, r4
    add r3, r3, #2
```

```
#elif LAB2_VARIANT == 2
    //  $r3 = ((a \& b) \gg 1) + c!$ 
    and r3, r0, r1
    lsr r3, r3, SHIFT_VALUE
    mov r0, r2
    bl factorial
    add r3, r3, r0
```

```
#elif LAB2_VARIANT == 3
    // if  $(a - b) \geq 0$ :  $r3 = (a + b) / c$ ; else  $r3 = (a - b) / c$ 
    sub r4, r0, r1
    cmp r4, #0
    blt variant3_negative
    add r3, r0, r1
    sdiv r3, r3, r2
    b variant_done
```

```
variant3_negative:
    sdiv r3, r4, r2
    b variant_done
```

```
#elif LAB2_VARIANT == 4
    // if  $(a * b) \geq 10$ :  $r3 = (a | b) * c$ ; else  $r3 = (a * b) * c$ 
    mul r4, r0, r1
    cmp r4, #10
    blt variant4_less
    orr r3, r0, r1
    mul r3, r3, r2
    b variant_done
```

```

variant4_less:
    mul r3, r4, r2
    b variant_done

#else
    // Bonus/example variant: r3 = ((a + c) << 1) - b
    add r3, r0, r2
    lsl r3, r3, #1
    sub r3, r3, r1
#endif

variant_done:
    pop {pc}

factorial:
    push {r1, r2, lr}
    mov r1, r0
    mov r0, #1
factorial_loop:
    cmp r1, #1
    ble factorial_end
    mul r0, r0, r1
    sub r1, r1, #1
    b factorial_loop
factorial_end:
    pop {r1, r2, pc}

```

Результат виконання

Після покрокового виконання у GDB результат обчислення фіксується у регістрі r3. Для використаних тестових значень $a = 14$, $b = 6$, $c = 4$ отримано значення $r3 = 26$.

Висновок

У лабораторній роботі реалізовано обчислення виразу мовою GAS для Cortex-M4, використано арифметичні, логічні, зсувні й умовні інструкції, а результат виконання винесено в регістр r3.

Лабораторна робота №3

Тема

Завантажувач основної програми, обробка виключень і вивід даних через Semihosting.

Мета роботи

Навчитися працювати з оперативною пам'яттю, векторною таблицею виключень, механізмом Semihosting та створити примітивний завантажувач, який переносить kernel.bin у RAM і передає йому керування.

Визначення варіанта

Варіант завантажувача визначається за формулою: номер залікової книжки $\% 16 =$ номер варіанта.

Підстановка значення: $1 \% 16 = 1$. Для виконання обрано варіант 1: LDR/STR, інкрементна адресація з регістровим зсувом, крок 4 байти.

Варіанти	Команди	Адресація	Крок
0-3	LDR / STR	інкремент або декремент	4 байти
4-7	LDRB / STRB	інкремент або декремент	1 байт

8-11	LDRH / STRH	інкремент або декремент	2 байти
12-15	LDM / STM	інкремент або декремент	8 або 12 байт

Хід виконання

Основна програма розміщена у файлі kernel.S. Вона містить власну векторну таблицю, виконує обчислення з лабораторної роботи №2 та виводить результат у консоль через Semihosting. Файл bootloader.S включає kernel.bin за допомогою директиви .incbin, копіює його в RAM та переходить до адреси обробника RESET, зчитаної з другої позиції векторної таблиці.

Параметри збірки для роботи: LAB2_VARIANT=1 і BOOT_VARIANT=1.

Фрагмент bootloader.S

```
.syntax unified
.cpu cortex-m4
.thumb

.global _bootload

#ifdef BOOT_VARIANT
#define BOOT_VARIANT 1
#endif

.section .rodata
image:
    .incbin "kernel.bin"
end_of_image:
str_boot_start: .asciz "bootloader started"
str_boot_end: .asciz "bootloader end"

.section .text
_bootload:
    ldr r0, =str_boot_start
    bl dbgput_line

    ldr r0, =end_of_image
    ldr r1, =image
    ldr r2, =_ram_start

#ifdef BOOT_VARIANT == 0
copy_loop:
    ldr r3, [r1], #4
    str r3, [r2], #4
    cmp r1, r0
    blo copy_loop

#elif BOOT_VARIANT == 1
    mov r5, #0
    mov r7, #4
copy_loop:
    ldr r3, [r1, r5]
    str r3, [r2, r5]
    add r5, r5, r7
    add r6, r1, r5
    cmp r6, r0
    blo copy_loop

#elif BOOT_VARIANT == 2
```

```
    sub r4, r0, r1
    add r1, r1, r4
    ldr r2, =_ram_start
    add r2, r2, r4
copy_loop:
    sub r1, r1, #4
    sub r2, r2, #4
    ldr r3, [r1]
    str r3, [r2]
    ldr r6, =image
    cmp r1, r6
    bhi copy_loop
```

```
#elif BOOT_VARIANT == 3
    sub r4, r0, r1
    add r1, r1, r4
    ldr r2, =_ram_start
    add r2, r2, r4
    mov r5, #4
copy_loop:
    sub r1, r1, r5
    sub r2, r2, r5
    ldr r3, [r1]
    str r3, [r2]
    ldr r6, =image
    cmp r1, r6
    bhi copy_loop
```

```
#elif BOOT_VARIANT == 4
copy_loop:
    ldrb r3, [r1], #1
    strb r3, [r2], #1
    cmp r1, r0
    blo copy_loop
```

```
#elif BOOT_VARIANT == 5
    mov r5, #0
    mov r7, #1
copy_loop:
    ldrb r3, [r1, r5]
    strb r3, [r2, r5]
    add r5, r5, r7
    add r6, r1, r5
    cmp r6, r0
    blo copy_loop
```

```
#elif BOOT_VARIANT == 6
    sub r4, r0, r1
    add r1, r1, r4
    ldr r2, =_ram_start
    add r2, r2, r4
copy_loop:
    sub r1, r1, #1
    sub r2, r2, #1
    ldrb r3, [r1]
    strb r3, [r2]
    ldr r6, =image
    cmp r1, r6
    bhi copy_loop
```

```
#elif BOOT_VARIANT == 7
    sub r4, r0, r1
    add r1, r1, r4
    ldr r2, =_ram_start
    add r2, r2, r4
    mov r5, #1
copy_loop:
    sub r1, r1, r5
    sub r2, r2, r5
    ldrb r3, [r1]
    strb r3, [r2]
    ldr r6, =image
    cmp r1, r6
    bhi copy_loop
```

Результати виконання

Після запуску в QEMU через консоль Semihosting виводяться повідомлення bootloader started, bootloader end, kernel started і рядок із числовим результатом у регістрі r3. Результат підтверджує завантаження kernel у RAM та передавання керування основній програмі.

Висновок

У лабораторній роботі реалізовано мінімальний завантажувач для Cortex-M4. Код демонструє роботу з пам'яттю, адресацією, векторною таблицею та Semihosting-виводом.

Загальний висновок

У комплекті лабораторних робіт реалізовано базовий цикл розробки для ARM Cortex-M4: створення мінімального асемблерного проекту, виконання обчислення за варіантом, побудову завантажувача, копіювання основної програми в оперативну пам'ять і передавання керування через вектор RESET. Для номера залікової книжки 1 використано варіант 1 у лабораторній роботі №2 та варіант 1 у лабораторній роботі №3.